ReFit: Repairing Speculative Drafters for Post-Trained Language Models

Anonymous ACL submission

Abstract

Speculative decoding with target-conditioned drafters, from autoregressive EAGLE-3 to block-diffusion DFlash, is among the fastest ways to serve LLMs, and a base model's released drafter is reused for its post-trained derivatives. A census of 174 public derivatives shows that reuse breaks for extensively post-trained reasoning models: quantized, merged and on-policy RL derivatives retain 98–104% of the drafter's acceptance, but reasoning distills lose up to 30%, erasing most of the speedup. Post-training shifts both the features the drafter reads and the tokens it must predict, so we introduce *ReFit*, which re-fits the family drafter on the derivative's own responses to public instructions, without original training data. On DeepSeek-R1-Distill-Llama-8B, re-fitting the warm-started drafter closes 72% of the gap to a dedicated drafter, and re-fitting only its *feature interface* closes 60%, each for two orders of magnitude less compute than a dedicated drafter's public recipe; ReFit raises the speedup from 1.34× to 1.85× on SPEED-Bench (dedicated: 2.08×) and from 1.49× to 2.32× on MATH-500, and on Llama-3.1-Nemotron-Nano-8B from 1.33× to 1.84×, outperforming independent draft models. ReFit is also, to our knowledge, the first method to repair block-diffusion drafters, raising DFlash's speedup to 2.26× on MATH-500.

Codebase will be released upon publication.

1 Introduction

Speculative decoding has become the standard way to serve large language models (LLMs) quickly: a small drafter proposes several tokens, and the target model verifies all of them in a single forward pass, accelerating generation without changing its output (Leviathan et al., 2023; Chen et al., 2023). The fastest drafters today are *target-conditioned*. Rather than modeling text from scratch, they read the target's own hidden states and predict its next tokens from them (Cai et al., 2024; Li et al., 2024b, 2025; Chen et al., 2026b). EAGLE-3, for instance, fuses features from the target's lower, middle and upper layers and reports speedups of up to 6.5× (Li et al., 2025), and DFlash drafts an entire block of tokens in one parallel pass of a lightweight diffusion model conditioned on the same features (Chen et al., 2026b). The price of this coupling is a costly recipe that is repeated for every target: regenerate a large corpus of the target's responses and train the drafter from scratch for several epochs (Li et al., 2026a; Red Hat AI and vLLM Project, 2025).

Open-weight models, however, rarely stay single models. A popular base model such as Llama-3.1-8B spawns thousands of derivatives on public model hubs, from quantized and merged variants to RL-tuned checkpoints and reasoning distillations (Horwitz et al., 2025). Training a dedicated drafter for each one is out of reach, so serving stacks reuse the drafter released for the family's instruction model, the *family drafter*, for all of its relatives (Kwon et al., 2023; Red Hat AI and vLLM Project, 2025). Two questions behind this practice remain open: when does a family drafter keep working for a derivative, and what can be done when it does not?

We answer the first question with a census of 174 public derivatives in the Llama-3.1-8B and Qwen3-8B families (Figure 1a). For most of the ecosystem, reuse is reliable: quantized, merged, abliterated and on-policy RL derivatives retain 98–104% of the family drafter's acceptance, in line with evidence that on-policy training keeps models close to their initialization (Shenfeld et al., 2026; Chen et al., 2026a). Reuse fails, however, for precisely the derivatives that are most expensive to serve: reasoning models produced by extensive off-policy post-training, which retain only 70–86%. The failure does not depend on the drafter's design: DeepSeek-R1-Distill-Llama-8B and Llama-3.1-Nemotron-Nano-8B lose 26–30% of first-token

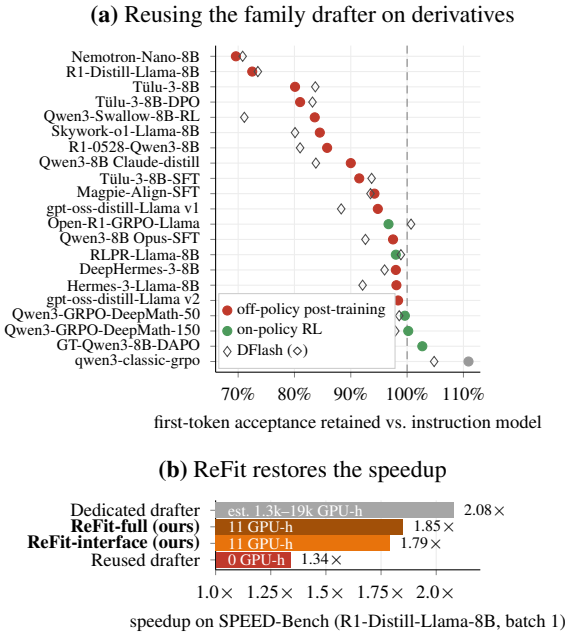


Figure 1: **(a)** First-token acceptance retained by the production family drafters (EAGLE-3 dots, DFlash diamonds) on 21 widely used Llama-3.1-8B and Qwen3-8B derivatives, on the same 128 SPEED-Bench prompts: reuse fails for the extensively post-trained reasoning models at the top (174-derivative census in Appendix F). **(b)** On R1-Distill-Llama-8B, ReFit recovers most of a dedicated drafter's speedup at a small fraction of its training cost.

acceptance under both autoregressive EAGLE-3 and block-diffusion DFlash drafters. For R1-Distill, the speedup collapses from 2.08× with a dedicated drafter to 1.34× (Figure 1b), and reasoning models emit thousands of tokens per query, so the loss costs most exactly where acceleration matters most.

We then show what changes and how to repair it. Post-training changes two things the drafter depends on: the target's hidden states, which the drafter reads, and the target's next-token policy, which it must predict. A controlled crossover shows that the hidden states shift even on text the derivative did not write: the drafter faces new text, encoded differently and continued differently. The drafter accesses the target through a single learned map, the *feature interface*, a dense projection that compresses the target's multi-layer hidden states into the drafter's input space, and re-fitting this interface alone recovers most of what re-training the entire drafter achieves. This observation leads to *ReFit* (Figure 2): elicit the derivative's own responses to public instructions, then re-fit the family drafter on them with its native training objective, either the whole warm-started drafter (ReFit-full) or only its interface (ReFit-interface), a lightweight update that leaves the rest of the drafter untouched. ReFit needs no access to the derivative's or the drafter's training data, no new architecture, and no change to the serving engine. ReFit departs from the standard per-target recipe, which regenerates hundreds of thousands of responses and trains a new drafter from scratch for many epochs (Li et al., 2026a; Red Hat AI and vLLM Project, 2025): it starts from the released drafter and needs 16k responses and a single epoch, a budget at which training from scratch does not even match the unrepaired drafter. Unlike prior drafter adaptation, which designs new drafter architectures (Lin et al., 2026), restricts how the target may be fine-tuned (Li et al., 2026b), or tracks a fixed target online (Liu et al., 2024; Zhang et al., 2026), ReFit repairs the drafters already deployed, in their released format, offline and after arbitrary third-party post-training, for autoregressive and block-diffusion drafters alike.

ReFit is highly effective. On R1-Distill-Llama-8B, ReFit-full closes 72% of the gap between the reused family drafter and a dedicated drafter trained by the EAGLE authors, and ReFit-interface closes 60%; with 64k repair examples, these rise to 79% and 66%. With 11 GPU-hours of compute, two orders of magnitude below the estimated cost of a dedicated drafter, ReFit lifts the end-to-end speedup from 1.34× to 1.85× on SPEED-Bench and from 1.49× to 2.32× on MATH-500. On Nemotron-Nano, for which no dedicated drafter exists, it lifts the speedup from 1.33× to 1.84×. ReFit also repairs block-diffusion drafters, raising DFlash's acceptance length by 23–29% and its speedup to up to 2.26×. Repaired drafters are also faster than independent small draft models and training-free drafting. Our contributions are:

- The first systematic census of family-drafter reuse, across 174 public derivatives, showing that reuse fails under extensive off-policy post-training, above all in today's reasoning models, and a 16-request probe that detects such failures (§3).
- ReFit, a simple recipe that repairs the family drafter from the derivative's own outputs without any original training data; re-fitting the feature interface alone recovers most of the loss (§4).
- To our knowledge, the most extensive eval-

uation of drafter adaptation to date, with wall-clock timing throughout, across targets, model families, drafter checkpoints, reasoning modes, sampled decoding and long traces, and the first study of block-diffusion drafters under target post-training (§6).

2 Background: Target-Conditioned Drafters

Speculative decoding. Given a prefix $y_{\le t}$, a drafter proposes K tokens $\hat{y}_{t+1:t+K}$. The target scores all of them in one forward pass and accepts the longest prefix that agrees with its own predictions, plus one bonus token that it generates itself (Leviathan et al., 2023; Chen et al., 2023). The output is identical to decoding with the target alone. With acceptance length τ, the average number of tokens produced per target forward pass, and per-step costs c_T for the target and c_D for the drafter, the speedup is approximately $\tau\, c_T/(c_T + Kc_D)$, so a good drafter is both *accurate* and *cheap*.

Target-conditioned drafting. Target-conditioned drafters achieve both properties by reusing the target's computation. EAGLE-3 (Li et al., 2025) reads hidden states $h_t^{\text{low}}, h_t^{\text{mid}}, h_t^{\text{high}} \in \mathbb{R}^d$ from three depths of the target and maps them through a learned *feature interface* $W_{\text{fc}} \in \mathbb{R}^{d\times 3d}$:

$$g_t = W_{\text{fc}}\,[\,h_t^{\text{low}};\, h_t^{\text{mid}};\, h_t^{\text{high}}\,]. \tag{1}$$

The fused feature g_t is combined with the embedding of the next token and passed through a single decoder layer and a language-modeling head over a reduced draft vocabulary, producing the draft distribution q_θ. EAGLE-3 is trained with *training-time test*: the drafter is unrolled for $k = 1, \ldots, k_{\max}$ steps, feeding its own predicted features forward as it does at inference, and matched at each step to the target's next-token distribution renormalized over the draft vocabulary, $\tilde{p}_T$:

$$\mathcal{L}(\theta) = \sum_{k=1}^{k_{\max}} \mathbb{E}_t\, \mathrm{KL}\Big(\tilde{p}_T(\cdot \mid y_{\le t+k}) \,\Big\|\, q_\theta^{(k)}\Big). \tag{2}$$

DFlash (Chen et al., 2026b) drafts an entire block in parallel with a lightweight diffusion model, but it reads the target through the same kind of multi-layer feature projection.

Family drafters. A drafter is trained once for a family's instruction model and released publicly. Serving engines then load it for any model that shares that model's architecture and tokenizer (Kwon et al., 2023; Red Hat AI and vLLM Project, 2025). We call the instruction model the *parent* T, its drafter the *family drafter* D_{θ_0}, and the model being served the *derivative* $\mathcal{T}'$.

3 When Family Drafters Fail

A census of derivatives. We collect 174 public derivatives in the Llama-3.1-8B and Qwen3-8B families (Grattafiori et al., 2024; Yang et al., 2025) from the Hugging Face hub, all of which load with the architecture and tokenizer of the instruction-tuned parent and its released drafters, and label each by the post-training history described in its model card. For every derivative, we serve the production EAGLE-3 and DFlash family drafters with vLLM (Kwon et al., 2023) and measure first-token acceptance on identical prompts for the derivative and its parent; we report retention, the derivative's acceptance divided by the parent's. We measure 163 derivatives on prompts from their own domain and the other 11 on general prompts (Appendix F), and a focused cohort of 21 widely used post-trained derivatives on the same 128 SPEED-Bench prompts (Figure 1a).

Reuse fails under extensive off-policy post-training. Edits in weight space leave the drafter intact: pooled over both families, quantized, merged and abliterated derivatives retain 98–100% of the parent's acceptance under both drafters (Appendix F). On-policy RL does not disturb it either: RL-tuned derivatives retain 104% of EAGLE-3's and 100% of DFlash's acceptance on average, and the five in Figure 1a retain 97–103% of EAGLE-3's. Off-policy post-training is different. Most off-policy fine-tunes, including several small reasoning-distillation fine-tunes, retain 90–98% under EAGLE-3, but the most extensively post-trained models lose far more: large-scale reasoning distills (R1-Distill-Llama-8B, R1-0528-Qwen3-8B) and multi-stage pipelines (Nemotron-Nano-8B, Tülu 3, Skywork-o1, Qwen3-Swallow) retain only 70–86%. Along the Tülu 3 pipeline (Lambert et al., 2024), the off-policy stages lower retention (SFT 92%, preference optimization 81%), while the final on-policy RL stage leaves it unchanged (80%). This ordering matches how the two regimes are known to move a model: on-policy RL stays close to its initialization (Shenfeld et al., 2026; Chen et al., 2026a), whereas off-policy training fits it to another distribution (Hinton et al., 2015; Kim and

Rush, 2016).

The cost is substantial: on R1-Distill, the reused drafter reaches only 1.76 tokens per step on SPEED-Bench, against 2.85 for a dedicated drafter trained by the EAGLE authors, in line with a concurrent observation (Jiang et al., 2026), and most derivatives, including Nemotron-Nano, have no dedicated drafter at all. The failure is also cheap to detect: retention on just 16 requests identifies degraded derivatives with an AUROC of 0.91–0.93 (Appendix F).

What changes. A derivative differs from its parent in the text it writes, in how it encodes text, and in the token it wants next. New text alone does not break the drafter: RL-trained reasoning derivatives, which write long chains of thought, keep 97–103% of their EAGLE-3 acceptance. What moves is the model. In a fixed-text crossover, we feed the family drafter either the parent's or the derivative's hidden states on identical text and score its first prediction against either model's next token. The derivative's hidden states lower agreement even on the parent's own outputs, by 4.3 points on R1-Distill and 9.2 on Nemotron-Nano, and similarly on public reference text (95% CIs exclude zero), whereas an RL-tuned derivative shows no shift (0.1 points); on the derivatives' own text, their next-token policy lowers agreement by a further 3.0 and 3.7 points. The family drafter therefore faces new text, encoded in shifted hidden states and continued by a different policy, and a repair has to learn all three from the derivative itself.

A single module carries most of the repair. Which part of the drafter needs to change? Refitting different parts of the R1-Distill production drafter on 256 of the derivative's responses (Figure 3) shows that recalibrating the mean and scale of the target's features closes none of the gap to the dedicated drafter, whereas re-fitting the dense feature interface alone closes 36%, most of what re-training the whole decoder layer (43%) or the entire drafter (46%) achieves with a fifth to an eighth of the parameters. The entire warm-started drafter and its interface therefore define the two operating points of ReFit.

4 ReFit: Repairing the Family Drafter

Given a derivative $\mathcal{T}'$ and the family drafter D_{θ_0} of its parent, ReFit produces a drafter D_θ matched to $\mathcal{T}'$ without the data used to train either model. The recipe has three steps (Figure 2).

Eliciting the derivative. The drafter must predict what $\mathcal{T}'$ will generate, so we let $\mathcal{T}'$ show it. We sample 16k instructions x from the public Alpaca corpus (Taori et al., 2023), deduplicated against all evaluation prompts; instructions elicited from the derivative itself, which avoid any external dataset and its license, give the same SPEED-Bench acceptance (Appendix A). We render each instruction with the derivative's chat template and greedily decode a response $y' \sim \mathcal{T}'(\cdot \mid x)$ of up to 512 tokens. For a reasoning distill, these responses are long reasoning traces, exactly the text the drafter will face in deployment.

Re-fitting the drafter. For each pair (x, y'), a single teacher-forced pass of $\mathcal{T}'$ provides, at every response position, the three hidden states that enter Eq. (1) and the next-token distribution $p_{\mathcal{T}'}$. These two signals address exactly the two shifts identified in §3: the drafter learns to read $\mathcal{T}'$'s features and to predict $\mathcal{T}'$'s tokens. We minimize the drafter's native training-time-test objective (Eq. 2) with $\tilde{p}_{\mathcal{T}'}$ in place of $\tilde{p}_T$ and $k_{\max} = 3$, starting from the family drafter's parameters θ_0. We consider two operating points: **ReFit-full**, our default, warm-starts all drafter parameters from θ_0, with the token embedding shared with $\mathcal{T}'$; **ReFit-interface** updates only W_{fc}, 50M parameters or 12% of the drafter, and leaves the decoder layer and head untouched. Both variants inherit everything the family drafter already knows about drafting, and this warm start is essential (§6).

Cost and deployment. ReFit takes one pass over 16k examples (7.9M response tokens), about 11 A40 GPU-hours in total, of which training the drafter accounts for only 2.2. The repaired drafter keeps the family drafter's architecture, vocabulary and file format and, unlike expert-based adaptation (Lin et al., 2026), adds no parameters or operators, so it is a drop-in replacement in the serving engine. ReFit-full gives the largest speedups; ReFit-interface suits providers that serve many derivatives of one family, because every interface-repaired drafter keeps the released decoder layer and head bit for bit and differs from the release only in a 100 MB matrix, rather than an 850 MB drafter.

5 Experimental Setup

Targets and drafters. We study the two reasoning distills from the census: DeepSeek-R1-

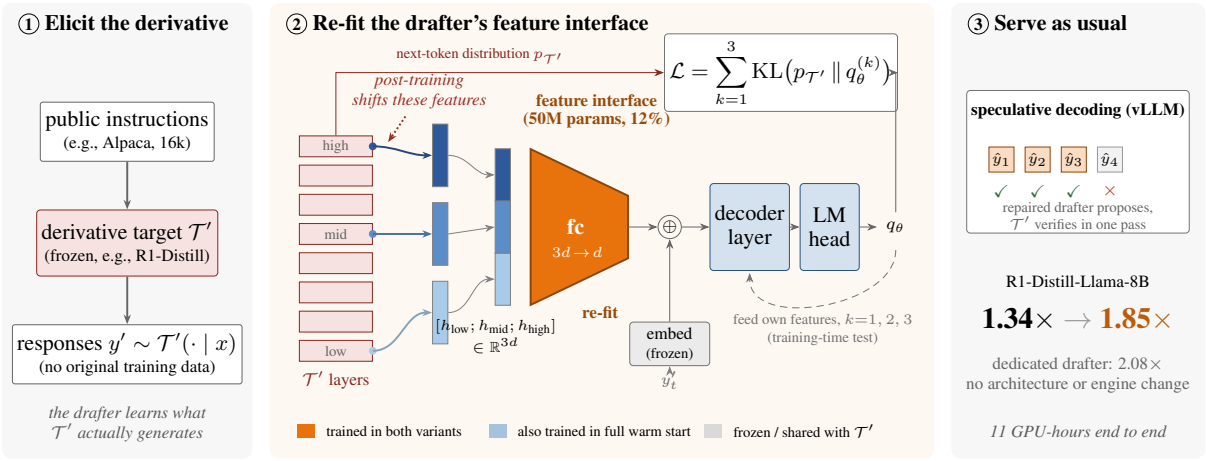


Figure 2: **ReFit.** ① The derivative target $\mathcal{T}'$ answers public instructions; none of its training data, nor the drafter's, is needed. ② A teacher-forced pass of $\mathcal{T}'$ exposes the hidden states the drafter reads and the next-token distribution it must match; post-training has shifted both, so we re-fit the drafter's feature interface (orange, ReFit-interface) with its native training-time-test objective, optionally together with the decoder layer and head (blue, ReFit-full). ③ The repaired drafter keeps its architecture and format and is served with an unchanged engine.

Distill-Llama-8B, distilled from DeepSeek-R1 into Llama-3.1-8B (DeepSeek-AI, 2025), and Llama-3.1-Nemotron-Nano-8B-v1, derived from Llama-3.1-8B-Instruct through distillation and reasoning post-training (Bercovich et al., 2025). Both share the Llama-3.1 architecture and tokenizer, and hence the family drafters of Llama-3.1-8B-Instruct. Our main family drafter is the EAGLE-3 drafter that the EAGLE authors released for Llama-3.1-8B-Instruct (Li et al., 2025). We also repair the production EAGLE-3 drafter distributed with vLLM's speculators library (Red Hat AI and vLLM Project, 2025) and the DFlash drafter (Chen et al., 2026b). As an upper reference, we use the dedicated EAGLE-3 drafter that the EAGLE authors trained specifically for R1-Distill-Llama-8B. Nemotron-Nano is served in its default chat mode and, in §6.2, in its reasoning mode. For a second model family, we repair the Qwen3-8B family EAGLE-3 drafter on DeepSeek-R1-0528-Qwen3-8B.

Baselines. We compare against reusing the family drafter unchanged; an independent draft model, Llama-3.2-1B-Instruct (Grattafiori et al., 2024), which shares the targets' tokenizer; n-gram prompt lookup (Saxena, 2023), built into vLLM; the same EAGLE-3 architecture trained from scratch on our repair data; LoRA adapters on every drafter layer (Hu et al., 2022) with as many trainable parameters as the interface; and training-free feature alignment, which folds a per-layer linear map from the derivative's to the parent's hidden states into the interface. All speculative methods propose $K{=}4$ tokens per step.

Benchmarks and metrics. We evaluate on SPEED-Bench (Abramovich et al., 2026), using a stratified 128-prompt sample of its qualitative split, which spans eleven domains from coding and math to writing, roleplay and multilingual chat. For mathematical reasoning, we use MATH-500 (Hendrycks et al., 2021; Lightman et al., 2024), and for long reasoning, MATH problems with responses of up to 8,192 tokens. Prompts are rendered with each target's default chat template, and responses are generated greedily for up to 512 tokens unless stated otherwise. We report first-token acceptance p_1, the rate at which the target accepts the first drafted token, τ (including the bonus token) and, for R1-Distill, the *gap closed*, $(\tau_{\text{repair}} - \tau_{\text{reuse}})/(\tau_{\text{dedicated}} - \tau_{\text{reuse}})$. We measure end-to-end wall-clock speedup over autoregressive decoding in vLLM on NVIDIA A40 GPUs at batch sizes 1 and 8. R1-Distill acceptance results average three training seeds; confidence intervals are 95% paired bootstrap intervals over prompts (and seeds). Further details are in Appendix A.

6 Results

6.1 Main Results

Table 1 summarizes our results. On R1-Distill-Llama-8B, ReFit-interface raises the acceptance length on SPEED-Bench from 1.76 to 2.41 and ReFit-full to 2.54 (paired gains

		R1-Distill-Llama-8B					Nemotron-Nano-8B				
	Train	p_1 (%)	τ		Speedup		p_1 (%)	τ		Speedup	
Method	GPU-h	SPEED	SPEED	MATH	bs 1	bs 8	SPEED	SPEED	MATH	bs 1	bs 8
Autoregressive	–	–	1.00	1.00	1.00×	1.00×	–	1.00	1.00	1.00×	1.00×
Independent drafter (1B)	–	61.3	2.44	2.97	1.29×	1.11×	64.7	2.65	2.79	1.43×	1.24×
Family drafter, reused	0	43.0	1.76	1.93	1.34×	1.07×	42.9	1.76	1.69	1.33×	1.13×
ReFit-interface (ours)	11	62.6	2.41	2.78	1.79×	1.27×	61.4	2.42	2.66	1.74×	1.39×
ReFit-full (ours)	11	65.0	2.54	2.97	**1.85**×	**1.29**×	62.9	2.49	2.86	**1.84**×	**1.42**×
Dedicated drafter	1.3k–19k	70.5	2.85	3.84	2.08×	1.41×	*not available*				

Table 1: **Main results.** First-token acceptance p_1, acceptance length τ and end-to-end speedup over autoregressive decoding (vLLM, A40); MATH is MATH-500, and speedups are on SPEED-Bench (MATH-500 in Table 2; CIs in Appendix D). Every acceptance gain over reuse has a 95% CI excluding zero. Training cost includes data generation; the dedicated drafter's is a recipe estimate for 10–40 epochs of the public EAGLE-3 recipe.

$0.650\,[0.607,\ 0.692]$ and $0.778\,[0.731,\ 0.823]$), closing 60% and 72% of the gap to the dedicated drafter; ReFit-full brings first-token acceptance within 5.5 points of it. On MATH-500, ReFit raises τ from 1.93 to 2.78 and 2.97. On Nemotron-Nano, where no dedicated drafter exists and repair is the only way to obtain a matched drafter, τ rises by 0.66 and 0.72 on SPEED-Bench and by 0.98 and 1.17 on MATH-500, and both variants improve all eleven SPEED-Bench domains on both targets (Appendix E).

These gains carry over to wall-clock time. At batch size 1, ReFit-interface and ReFit-full accelerate R1-Distill by 1.79× and 1.85×, against 1.34× for the reused drafter and 2.08× for the dedicated one, closing 69% of the wall-clock gap. On Nemotron-Nano, the speedup rises from 1.33× to 1.84×. Measured directly against reuse on the same prompts and server processes, ReFit-full decodes 1.39× and 1.38× faster (95% CIs $[1.35, 1.43]$ and $[1.33, 1.44]$). At batch size 8, where speculative decoding has less idle compute to exploit, ReFit still lifts the speedup from 1.07× to 1.29× on R1-Distill and from 1.13× to 1.42× on Nemotron-Nano.

What makes ReFit work. The first of three ingredients is the warm start. The same EAGLE-3 architecture trained from scratch on the repair data reaches only 1.52 tokens per step on SPEED-Bench, below even the unrepaired family drafter, and even with four times as much data and 10 GPU-hours of training it only matches the reused drafter (1.74 versus 1.76; 1.97 versus 1.93 on MATH-500), while ReFit-full reaches 2.54 and 2.97 from 16k examples. A drafter's ability to draft is expensive to acquire, and the family drafter already has it. The second is supervision from the derivative itself. Training on the same 16k derivative-generated responses, but with the parent's hidden states and next-token distributions as supervision, recovers 54–62% of the gain, because the drafter adapts to the new text. Supervision from the derivative supplies the rest: 0.29–0.31 further tokens per step on SPEED-Bench and 0.39–0.40 on MATH-500, with every 95% CI above 0.25. New text does not break the drafter by itself (§3), but once the target has moved, the repaired drafter must learn both its text and its signals, which is exactly what eliciting the derivative provides. The third is a full-parameter update: LoRA adapters on every drafter layer, with as many trainable parameters as the interface, fall short of ReFit-full by 0.09 tokens per step on R1-Distill and 0.06 on Nemotron-Nano at a similar training cost.

ReFit versus other drafters. N-gram lookup stays below 1.6 tokens per step. The independent Llama-3.2-1B drafter is as accurate as ReFit-interface on R1-Distill (2.44 versus 2.41 tokens per step) but runs sixteen layers per drafted token, which holds it to 1.29× against 1.79×: at equal acceptance, the near-free drafting of a target-conditioned drafter decides the speedup. Training-free feature alignment, a per-layer linear map from the derivative's to the parent's hidden states folded into the interface, gains at most 0.17 tokens per step and loses accuracy on Nemotron-Nano. Repaired drafters combine high acceptance with near-free drafting and are the fastest option on both targets and at both batch sizes.

6.2 Reasoning Workloads

Reasoning models are deployed on mathematical problems with long generation budgets and often

	R1-Distill			Nemotron-Nano		
	MATH-500		8k	MATH-500		8k
Method	bs 1	bs 8	bs 1	bs 1	bs 8	bs 1
Reused	1.49	1.33	1.33	1.30	1.36	1.12
Independent (1B)	1.58	1.39	–	1.50	1.50	–
ReFit-interface	2.15	1.93	1.63	2.07	2.12	1.54
ReFit-full	**2.32**	**2.05**	**1.77**	**2.22**	**2.23**	**1.63**
Dedicated	2.94	2.54	2.11	–	–	–

Table 2: ReFit on reasoning workloads: speedup (×) over autoregressive decoding on MATH-500 (512-token responses) and on MATH problems with responses of up to 8,192 tokens.

	R1-Distill			Nemotron-Nano		
	Reuse	intf.	full	Reuse	intf.	full
τ, SPEED	2.05	2.39	**2.52**	2.08	2.44	**2.58**
τ, MATH-500	2.41	2.79	**3.00**	2.17	2.59	**2.80**
τ, MATH 8k	2.77	3.12	**3.37**	2.41	2.73	**2.91**
×, SPEED bs 1	1.54	1.77	**1.86**	1.55	1.78	**1.89**
×, SPEED bs 8	1.23	1.38	**1.44**	1.34	1.56	**1.62**
×, MATH bs 1	1.79	2.10	**2.26**	1.65	1.96	**2.11**
×, MATH bs 8	1.53	1.78	**1.93**	1.43	1.68	**1.79**

Table 3: ReFit repairs the block-diffusion drafter DFlash (intf.: ReFit-interface). τ: acceptance length (three seeds); ×: speedup. All gains over reuse have 95% CIs excluding zero.

with sampling. Table 2 shows that ReFit's absolute speedups are largest exactly there. On MATH-500, ReFit-full accelerates R1-Distill by 2.32× at batch size 1 and 2.05× at batch size 8, compared with 1.49× and 1.33× for the reused drafter, and Nemotron-Nano by 2.22× and 2.23×. With responses of up to 8,192 tokens, 2.4k–3.3k on average, ReFit-full still delivers 1.77× and 1.63×, against 1.33× and 1.12× for reuse; measured directly against reuse on the same problems and processes, it decodes 1.33× and 1.47× faster (95% CIs $[1.15, 1.62]$ and $[1.28, 1.65]$). On 128 such problems, ReFit-full raises the acceptance length from 1.90 to 2.65 on R1-Distill and from 1.62 to 2.58 on Nemotron-Nano (paired gains 0.76 $[0.68,\ 0.83]$ and 0.96 $[0.89,\ 1.03]$). Repair responses of at most 512 tokens suffice: for R1-Distill they cover only the opening of its reasoning traces (72% reach the cap), yet repairing on responses of up to 2,048 tokens changes acceptance on these long traces by at most 0.07 tokens per step. Its gains also hold under temperature sampling (T=0.6), where verification accepts tokens stochastically: on these long traces, ReFit-full raises the acceptance length from 1.96 to 2.58 on R1-Distill and from 1.64 to 2.63 on Nemotron-Nano (Appendix B).

Reasoning mode. In Nemotron-Nano's explicit reasoning mode, the reused drafter reaches only 1.70 tokens per step on MATH-500; the drafter repaired on default-mode responses raises this to 2.86 and decodes 1.68× faster than reuse (95% CI $[1.66, 1.71]$), and eliciting the repair data in reasoning mode reaches 2.87 and 1.69× (Appendix B).

6.3 Block-Diffusion Drafters

DFlash (Chen et al., 2026b) replaces token-by-token drafting with a single parallel pass of a lightweight diffusion model that proposes a block of ten tokens. It is recent enough that no prior work has studied it under target post-training, yet it reads the target through the same kind of feature interface and breaks the same way under reuse, retaining only 73.5% and 70.8% of its first-token acceptance on R1-Distill and Nemotron-Nano. We apply ReFit unchanged, with the same 16k responses and DFlash's own block objective, and it repairs DFlash on every benchmark and at every batch size (Table 3). ReFit-full raises DFlash's acceptance length by 23% and 24% on SPEED-Bench and by 24% and 29% on MATH-500, and adds 0.61 and 0.49 tokens per step on reasoning traces of up to 8,192 tokens. Measured directly against reuse on the same prompts, the repaired drafter decodes 1.21–1.28× faster at batch size 1 and 1.18–1.26× at batch size 8, reaching 2.26× over autoregressive decoding on MATH-500. ReFit-interface, which re-fits only DFlash's 84M-parameter interface, delivers most of this gain, and repair takes at most 1.5 GPU-hours of training. To our knowledge, this is the first adaptation of a block-diffusion drafter to a post-trained target; both the failure and its repair are properties of target-conditioned drafting, not of one drafter design.

6.4 Scaling and Cost

ReFit improves steadily with data (Figure 4 in Appendix C). With the EAGLE authors' drafter, the gap closed by ReFit-full grows from 53% with only 0.5k examples, 0.4 GPU-hours including data generation, to 64% with 4k, 72% with 16k and 79% with 64k, where the acceptance length reaches 2.62 on SPEED-Bench and 3.07 on MATH-500 and the batch-size-1 speedup 1.90×, close to the dedicated drafter's 2.08×. ReFit-interface follows a parallel curve (40% to 66%), and the production drafter

shows the same steady trend (46% to 74%).

Measured in serving time, the repair pays for itself quickly: at batch size 1, the 11 GPU-hours of ReFit-full are recovered after about 13.6k requests like those in SPEED-Bench (6.8M generated tokens) relative to the reused drafter, or 11.7k like those in MATH-500. By contrast, reproducing the dedicated drafter with the public EAGLE-3 recipe would take an estimated 1.3k–19k A40 GPU-hours including data generation, so ReFit recovers most of its speedup for 120 to 1,700 times less compute.

6.5 Generality

ReFit does not depend on a particular drafter checkpoint or model family (Table 5, Appendix C). The production EAGLE-3 drafter distributed with vLLM is repaired almost as well as the EAGLE authors' drafter, closing 66% of the R1-Distill gap, and the Qwen3 family drafter, reused on DeepSeek-R1-0528-Qwen3-8B, gains 0.38 tokens per step on SPEED-Bench and 0.47 on MATH-500, raising its speedup from 1.55× to 1.85×, ahead of an independent Qwen3 draft model (1.52×). With every drafter, ReFit-interface alone delivers most of ReFit-full's gain.

7 Related Work

Target-conditioned drafters and their training. Beyond model-free and self-speculative drafting (Fu et al., 2024; He et al., 2024; Saxena, 2023; Oliaro et al., 2024; Zhang et al., 2024a; Miao et al., 2024; Chen et al., 2024; Xia et al., 2024), the strongest drafters condition on the target's hidden states (Cai et al., 2024; Ankner et al., 2024; Li et al., 2024b,a, 2025; Zhang et al., 2024b; Jiang et al., 2026; Chen et al., 2026b), trained per target from scratch (Li et al., 2026a; Red Hat AI and vLLM Project, 2025). Recent work refines such drafters for a fixed target (Lei et al., 2026; Eldenk et al., 2026; EAGLE Team et al., 2026; Zhao et al., 2025; Goel et al., 2025); ReFit applies to them as released.

Aligning drafters to a target. DistillSpec (Zhou et al., 2024), direct alignment (Goel et al., 2024), domain draft models (Hong et al., 2025) and OmniDraft (Ramakrishnan et al., 2025) distill standalone token-level draft models toward a target, often from target-generated responses; ReFit instead repairs a released target-conditioned drafter, whose input, the target's hidden states, shifts as well. Online methods instead update drafters on live traffic for a fixed target (Liu et al., 2024; Zhang et al., 2026; Wang et al., 2026) or track a gradually changing policy during RL (Chen et al., 2025; Zhang et al., 2025); ReFit handles a single large shift offline, from the released drafter and the derivative's weights alone.

Adapting drafters when the target changes. The closest work to ours is EDA (Lin et al., 2026), which designs a drafter for adaptation: it replaces the feed-forward layers of an EAGLE-style drafter with gated shared and private experts, the latter modeling each target, and trains the private experts on target-regenerated responses selected by a representation-based criterion, for math, code and medical fine-tunes of one base model. ReFit addresses a different problem with a different mechanism. It repairs the drafters that are already released and deployed, in place: neither variant adds parameters, experts or routing, so the repaired drafter keeps the released checkpoint structure and runs in an unchanged serving engine. Where EDA adds target-specific capacity inside the decoder, ReFit-interface re-fits only the projection through which the drafter reads the target, because post-training shifts the very hidden states that this projection reads (§3), and leaves the decoder and head bit-identical to the release. ReFit applies, with each drafter's native objective, to multi-layer EAGLE-3 and block-diffusion DFlash drafters after arbitrary third-party post-training, including the extensive reasoning pipelines where reuse fails most, and our census establishes when such repair is needed. FlexSpec (Li et al., 2026b) instead keeps a frozen drafter compatible by restricting the target to parameter-efficient fine-tuning. Like EDA, we find adaptation far more effective than re-training from scratch.

8 Conclusion

Family drafters lose much of their value on extensively post-trained derivatives; re-fitting them on the derivative's own responses restores most of a dedicated drafter's speedup for a few GPU-hours, for autoregressive and block-diffusion drafters alike: when the target moves, the drafter can learn to follow it from the target itself.

Limitations

Our study focuses on 8B-scale derivatives of the Llama-3.1 and Qwen3 families and on the EAGLE-3 and DFlash drafters, which are among the most

widely deployed configurations. ReFit requires running the derivative to generate responses; this cost is small and is included in all our compute figures. We measure wall-clock speedups on NVIDIA A40 GPUs in vLLM with chain drafting, the configuration supported by our serving engine; absolute speedups vary with hardware, batch size and decoding configuration.

Ethical Considerations

ReFit leaves the target model's weights and outputs unchanged: speculative decoding is lossless, so repair affects only serving cost, not what the model generates. By reducing the compute needed to serve reasoning models, it lowers the energy cost of deploying them. All models and prompt sources are public and used under their licenses, and no private user data is involved.

References

Talor Abramovich, Maor Ashkenazi, Izzy Putterman, Benjamin Chislett, Tiyasa Mitra, Bita Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. 2026. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *International Conference on Machine Learning (ICML)*.

Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. 2024. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*.

Akhiad Bercovich et al. 2025. Llama-Nemotron: Efficient reasoning models. *arXiv preprint arXiv:2505.00949*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. 2026a. Retaining by doing: The role of on-policy data in mitigating forgetting. In *International Conference on Machine Learning (ICML)*.

Jian Chen, Yesheng Liang, and Zhijian Liu. 2026b. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*.

Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. 2025. ReSpec: Towards optimizing speculative decoding in reinforcement learning systems. *arXiv preprint arXiv:2510.26475*.

Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

EAGLE Team, vLLM Team, and TorchSpec Team. 2026. EAGLE 3.1: Advancing speculative decoding. https://lightseek.org/blog/eagle-3-1.html.

Doğaç Eldenk, Payal Mohapatra, Yigitcan Comlek, Kaan Oktay, Hongyang Zhang, and Stephen Xia. 2026. Attention drift: What autoregressive speculative decoding models learn. *arXiv preprint arXiv:2605.09992*.

Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of LLM inference using lookahead decoding. In *International Conference on Machine Learning (ICML)*.

Raghavv Goel, Mukul Gagrani, Wonseok Jeon, Junyoung Park, Mingu Lee, and Christopher Lott. 2024. Direct alignment of draft model for speculative decoding with chat-fine-tuned LLMs. *arXiv preprint arXiv:2403.00858*.

Raghavv Goel et al. 2025. VocabTrim: Vocabulary pruning for efficient speculative decoding in LLMs. *arXiv preprint arXiv:2506.22694*.

Aaron Grattafiori et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D. Lee, and Di He. 2024. REST: Retrieval-based speculative decoding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Fenglu Hong, Ravi Raju, Jonathan Lingjie Li, Bo Li, Urmish Thakker, Avinash Ravichandran, Swayambhoo Jain, and Changran Hu. 2025. Training domain draft models for speculative decoding: Best practices and insights. *arXiv preprint arXiv:2503.07807*.

Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. 2025. Charting and navigating Hugging Face's model atlas. *arXiv preprint arXiv:2503.10633*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Weifan Jiang, Krishna Teja Chitty-Venkata, Megan Flynn, Reed Meyerson, Zhenting Qi, Tianyu Wu, Eldar Kurtic, Minlan Yu, and Alexandre Marques. 2026. H-Spec: Parallel speculative decoding without a drafter-side KV cache. *arXiv preprint arXiv:2609.24197*.

Yoon Kim and Alexander M. Rush. 2016. Sequence-level knowledge distillation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. 2024. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*.

Haodi Lei, Yafu Li, Haoran Zhang, Shunkai Zhang, Qianjia Cheng, Xiaoye Qu, Ganqu Cui, Bowen Zhou, Ning Ding, Yun Luo, and Yu Cheng. 2026. Draft-OPD: On-policy distillation for speculative draft models. *arXiv preprint arXiv:2605.29343*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*.

Shenggui Li, Chao Wang, Yikai Zhu, Yubo Wang, Fan Yin, Shuai Shi, et al. 2026a. SpecForge: A flexible and efficient open-source training framework for speculative decoding. *arXiv preprint arXiv:2603.18567*.

Yuchen Li, Rui Kong, Zhonghao Lyu, Qiyang Li, Xinran Chen, Hengyi Cai, Lingyong Yan, Shuaiqiang Wang, Jiashu Zhao, Guangxu Zhu, Linghe Kong, Guihai Chen, Haoyi Xiong, and Dawei Yin. 2026b. FlexSpec: Frozen drafts meet evolving targets in edge-cloud collaborative LLM speculative decoding. *arXiv preprint arXiv:2601.00644*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024a. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024b. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*.

Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. 2026. Efficiently aligning draft models via parameter- and data-efficient adaptation. *arXiv preprint arXiv:2603.09527*.

Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Ion Stoica, Zhijie Deng, Alvin Cheung, and Hao Zhang. 2024. Online speculative decoding. In *International Conference on Machine Learning (ICML)*.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, et al. 2024. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick Qiao. 2024. SuffixDecoding: A model-free approach to speeding up large language model inference. *arXiv preprint arXiv:2411.04975*.

Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. 2025. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Red Hat AI and vLLM Project. 2025. Speculators: A unified library for speculative decoding algorithms in LLM serving. https://github.com/vllm-project/speculators.

Apoorv Saxena. 2023. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding.

Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. 2026. RL's razor: Why online reinforcement learning forgets less. In *International Conference on Learning Representations (ICLR)*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.

Junxiong Wang, Fengxiang Bie, et al. 2026. When RL meets adaptive speculative training: A unified training-serving system. *arXiv preprint arXiv:2602.06932*.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*.

An Yang et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024a. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*.

Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Ruiwen Xu. 2024b. Learning harmonized representations for speculative sampling. *arXiv preprint arXiv:2408.15766*.

Shuyu Zhang, Lingfeng Pan, Qicheng Wang, Yaqi Shi, Yueyang Tan, Ruyu Yan, Jiaqi Chen, Lixing Du, and Lu Wang. 2026. EvoSpec: Evolving speculative decoding via real-time vocabulary and parameter adaptation. *arXiv preprint arXiv:2605.27390*.

Yizhou Zhang, Ning Lv, Teng Wang, and Jisheng Dang. 2025. FastGRPO: Accelerating policy optimization via concurrency-aware speculative decoding and online draft learning. *arXiv preprint arXiv:2509.21792*.

Weilin Zhao et al. 2025. FR-Spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. *arXiv preprint arXiv:2502.14856*.

Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2024. DistillSpec: Improving speculative decoding via knowledge distillation. In *International Conference on Learning Representations (ICLR)*.

A Implementation Details

Repair. We optimize Eq. (2) with AdamW (learning rate 2×10^{-5}, weight decay 0.01, cosine schedule with 3% warm-up) for one epoch over the 16k repair examples, packing roughly 2k response tokens per step, with $k_{\max}$=3 training-time-test steps. The loss is computed on response positions only. The target's logits are restricted to the 32k draft vocabulary and renormalized before the KL divergence; at successive training-time-test steps the drafter consumes its own predicted features together with the observed response tokens. Repair data are generated greedily with vLLM using each target's default chat template, capped at 512 response tokens, and deduplicated exactly and approximately against all evaluation prompts, including the full MATH-500 set; 72% of R1-Distill's and 19% of Nemotron-Nano's responses reach the cap. ReFit-interface updates $W_{\text{fc}} \in \mathbb{R}^{4096\times12288}$ (50.3M parameters); ReFit-full updates the interface, the decoder layer and the language-modeling head (424.8M parameters), keeping the token embedding shared with the target. When a released drafter omits the embedding table, the trainer loads the target's embedding, exactly as the serving engine does. The three R1-Distill seeds use the same data and differ only in the training seed. Repair costs 11.1 A40 GPU-hours on R1-Distill (8.9 for generating responses, 2.2 for training) and 9.8–10.0 on Nemotron-Nano.

DFlash. We repair the released DFlash drafter for Llama-3.1-8B-Instruct with its native objective, a KL divergence over each drafted block with exponentially decaying position weights (γ=4) and up to 64 block anchors per packed sequence of 2,048 tokens, using the same 16k responses, optimizer, learning rate and schedule as for EAGLE-3. The target, token embedding and output projection stay frozen; ReFit-interface updates the 83.9M-parameter feature projection and ReFit-full all 1.05B trainable parameters. Training takes 1.26–1.50 A40 GPU-hours on R1-Distill and 0.82–1.08 on Nemotron-Nano, and acceptance results average three training seeds on both targets. No dedicated DFlash drafter for either target is publicly available.

Feature alignment baseline. For each of the three tapped layers, we fit a least-squares map and an orthogonal Procrustes map from the derivative's to the parent's hidden states on paired forward passes over 4k repair responses and fold them block-diagonally into W_{fc}; no parameter is trained, and fitting takes 0.3–0.5 GPU-hours. On SPEED-Bench and MATH-500, the orthogonal map changes τ by $+0.14$ and $+0.17$ on R1-Distill and by -0.08 and -0.06 on Nemotron-Nano; the least-squares map by -0.32 and -0.37, and -0.21 and -0.10.

Training resources. Over the same 200 training batches, ReFit-interface takes 1.57 s per step at 25.7 GiB peak memory, against 1.77 s and 29.1 GiB for ReFit-full. Its mutable state is the interface

alone: 100.7 MB in bf16, against 849.4 MB for the full drafter. The decoder layer and head of an interface-repaired drafter are bit-identical to the released family drafter.

Instruction source. Replacing Alpaca with 4k instructions elicited from the derivative itself, which requires no external dataset, leaves SPEED-Bench acceptance unchanged ($\Delta\tau = 0.004$ and 0.009 for ReFit-interface and ReFit-full, both within noise).

Draft vocabulary. ReFit keeps the family drafter's 32k draft vocabulary. Re-selecting the vocabulary from the derivative's responses before ReFit-full leaves acceptance unchanged on Nemotron-Nano (τ 2.47 versus 2.49 on SPEED-Bench, 2.85 versus 2.86 on MATH-500), so the inherited vocabulary is not a bottleneck.

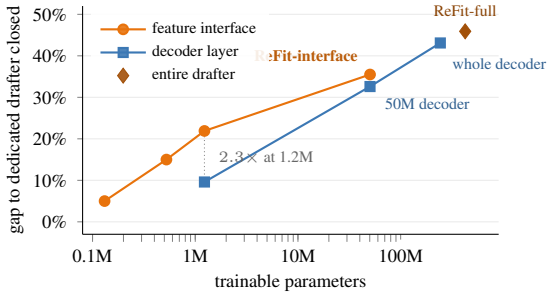


Figure 3: **Where to repair.** Fraction of the acceptance-length gap between the reused family drafter and a dedicated drafter closed by re-fitting the feature interface or the decoder layer with a given number of trainable parameters (R1-Distill-Llama-8B, production drafter, SPEED-Bench; same 256 repair examples and training recipe for every point). The dense interface (ReFit-interface) captures most of what re-training the entire drafter (ReFit-full) achieves, and with little data it outperforms decoder updates of equal size.

Component analysis. For Figure 3, all variants use the same 256 R1-Distill responses, batches, learning rate and 300 optimization steps, starting from the production EAGLE-3 drafter. Interface variants are LoRA updates of W_{fc} with ranks 8, 32 and 75 (0.13M, 0.52M and 1,228,800 parameters) and the dense interface (50,331,648). Decoder variants are LoRA on the query, value and MLP projections with rank 16 (1,228,800 parameters, exactly matching the rank-75 interface) and rank 655 (50.30M), a full-rank update of the query and output projections (50,331,648, exactly matching the dense interface), and the whole decoder layer (243.3M); the interface and head stay frozen in all decoder variants. The feature-statistics baseline fits a per-layer affine map that matches the mean and scale of the derivative's tapped features to the parent's. The parent-supervision control keeps the derivative-generated token sequences, masks, batches, initialization and objective fixed and replaces only the model that supplies the hidden states and next-token distributions. The whole-drafter LoRA baseline adapts every drafter linear layer (interface, attention, MLP and head) with rank 343 (50.31M parameters) on the 16k repair data.

Crossover. For each model, we take 64 contexts generated by the parent, by the derivative, and from public reference text, run both the parent and the derivative on the identical text, and feed each model's tapped hidden states to the frozen family drafter. Agreement is the rate at which the drafter's first prediction matches a given model's greedy next token. The feature effect averages the change in agreement from swapping parent features for derivative features under each fixed verifier; the policy effect averages the change from swapping the verifier under each fixed feature source. The two effects interact: on the derivatives' own text, agreement with derivative features and derivative verifier exceeds their additive prediction by 6.3 points on R1-Distill and 7.7 on Nemotron-Nano (95% CIs exclude zero; 64 sequences per text source), so the drafter's agreement depends jointly on whose features it reads and whose tokens it matches, and ReFit supervises both from the derivative.

Training-time-test depth. The drafter is unrolled for $k_{\max}$=3 steps during training and proposes K=4 tokens at serving time. Training with $k_{\max}$=4 instead gives the same acceptance ($\Delta\tau = -0.004$ and -0.007 on SPEED-Bench for ReFit-interface and ReFit-full at 4k examples, both within noise).

Evaluation. All acceptance measurements use vLLM with greedy decoding, K=4 draft tokens (DFlash: its native block of ten), a maximum of 512 new tokens and batch size 8. Nemotron-Nano is used with its default chat template, which carries no reasoning-mode instruction, except in the reasoning-mode experiments (Appendix B). Prompts that produce no speculative step are excluded pairwise across compared methods. Speedups are measured on 128 prompts (for MATH-500, the first 128 problems) and on 32 prob-

lems for responses of up to 8,192 tokens, each in three fresh server processes with three warm repetitions, relative to autoregressive decoding of the same prompts, using the seed-0 repair; confidence intervals resample processes and prompt batches jointly. The dedicated drafter's cost is a recipe estimate: the public EAGLE-3 recipe trains on 532k–646k examples, and our measured A40 throughput for generation and training projects 1.3k–19k GPU-hours at 10 to 40 epochs; the cost of the released dedicated checkpoint itself is not reported.

B Reasoning Workloads

The long-reasoning panel uses the first 32 problems of our 64-problem MATH sample, drawn from MATH-500, with response budgets of up to 8,192 tokens; the same engine, prompts and drafters are used as in the main results. Sampled decoding uses temperature 0.6 and top-p 0.95 with a fixed seed. No response in any arm exceeds 50% repeated 4-gram coverage. Table 4 reports acceptance lengths. On the first 128 MATH-500 problems with the same budget and greedy decoding, ReFit-interface and ReFit-full raise the acceptance length from 1.90 to 2.51 and 2.65 on R1-Distill (dedicated drafter: 3.49) and from 1.62 to 2.40 and 2.58 on Nemotron-Nano; every gain over reuse has a 95% confidence interval excluding zero. Repairing on 4k responses of up to 2,048 tokens, alone or mixed with the 16k short responses, changes acceptance on this panel by between -0.06 and $+0.07$ tokens per step relative to the short-response repair, for both targets and both variants (one seed). Reasoning mode adds Nemotron-Nano's *detailed thinking on* system turn to the same questions for every compared drafter; the mode-matched repair regenerates the 16k repair responses with this system turn. On traces of up to 8,192 tokens in reasoning mode, ReFit-full raises the acceptance length from 1.68 to 2.69, and to 2.73 with mode-matched data; on SPEED-Bench, from 1.74 to 2.46 and 2.49.

C Generality

Table 5 reports ReFit on every EAGLE-3 family drafter we repair; DFlash results are in Table 3. Figure 4 shows how the gap closed grows with the number of repair examples.

D Timing Details

Tables 6 and 7 report every wall-clock speedup with its 95% confidence interval, resampling server processes and prompt batches jointly, and Figure 5 relates speedup to training compute. On Nemotron-Nano MATH-500, the batch-size-8 speedups match or slightly exceed the batch-size-1 ones; batching changes the relative cost of target and drafter steps, and the two agree within their intervals.

			ReFit		
Target	Decoding	Reuse	interface	full	Dedicated
R1	greedy	1.97	2.59	**2.68**	3.78
	sampled	1.96	2.49	**2.58**	3.38
Nemotron	greedy	1.65	2.53	**2.73**	–
	sampled	1.64	2.42	**2.63**	–

Table 4: Acceptance length on MATH problems with responses of up to 8,192 tokens (2.4k–3.3k on average), under greedy decoding and temperature sampling. Every gain over reuse has a 95% confidence interval excluding zero.

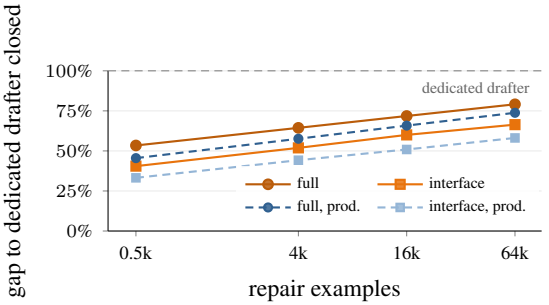


Figure 4: Gap to the dedicated drafter closed by ReFit-full (full) and ReFit-interface (interface) on R1-Distill-Llama-8B as a function of the number of repair examples, for the EAGLE authors' family drafter (solid) and the production drafter (dashed). The 64k points add Dolly instructions to Alpaca.

E Per-Domain Results

Table 8 reports the acceptance length on each of SPEED-Bench's eleven domains (11–12 prompts each). ReFit-full improves every domain on both targets, including multilingual chat, although the repair data contain only English instructions.

F Census Details

The census covers 174 derivatives, 87 in each of the Llama-3.1-8B and Qwen3-8B families, that load with the architecture and tokenizer of the instruction-tuned parent and its released production EAGLE-3 and DFlash drafters. Post-training histories are taken from model cards; models whose cards describe several stages are grouped by their combined history. For 163 derivatives, retention

Family drafter	Target	Reuse	ReFit interface	ReFit full
EAGLE-3 (authors)	R1	1.76	2.41	**2.54**
	Nemotron	1.76	2.42	**2.49**
EAGLE-3 (production)	R1	1.73	2.30	**2.47**
	Nemotron	1.82	2.32	**2.42**
EAGLE-3 (Qwen3)	R1-Qwen3	1.93	2.18	**2.30**

Table 5: ReFit repairs EAGLE-3 drafters across drafter checkpoints and model families. Acceptance length on SPEED-Bench with 16k repair examples. R1-Qwen3: DeepSeek-R1-0528-Qwen3-8B.

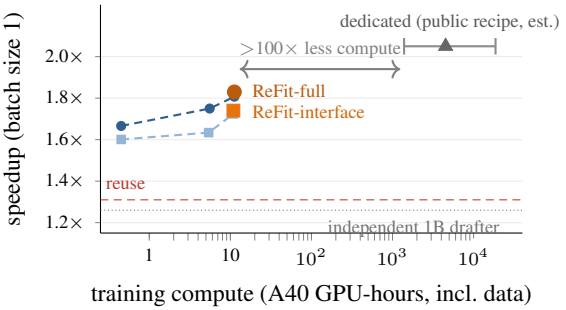


Figure 5: Batch-size-1 speedup on R1-Distill-Llama-8B against training compute, including data generation, on a 32-prompt SPEED-Bench panel. The dedicated drafter's range is a recipe estimate for 10 to 40 epochs of the public EAGLE-3 recipe.

is computed on 64 prompts from the derivative's own domain (Table 9); the remaining 11 use 128 general prompts. The reasoning distills and the controls discussed in §3 are evaluated on 128 SPEED-Bench prompts. Group means weight checkpoints equally, with hierarchical bootstrap intervals over checkpoints and paired prompts. Because each model is measured on its own responses to the same prompts, a derivative whose responses are easier to draft can exceed a retention of 1, as for several single-checkpoint groups in Table 9. Pooled over both families, mean retention under EAGLE-3 and DFlash is 1.00 and 1.00 for quantization (34 checkpoints; 95% CIs within $[0.99, 1.01]$), 0.99 and 0.98 for merges (7), 1.00 and 0.98 for abliteration (30), and 1.04 and 1.00 for on-policy RL (9). The compatibility probe ranks derivatives by acceptance retention on 16 fixed requests; its AUROC for identifying derivatives whose retention on the remaining, disjoint requests falls below 90% is 0.91 for EAGLE-3 (14 of 163 degraded) and 0.93 for DFlash (17 of 163), and 1.00 and 0.95 on a separate 21-checkpoint SPEED-Bench cohort.

Target	Method	SPEED-Bench bs 1	SPEED-Bench bs 8	MATH-500 bs 1	MATH-500 bs 8	MATH, 8k tokens bs 1
R1-Distill	Reused	1.34 [1.29, 1.38]	1.07 [0.99, 1.17]	1.49 [1.47, 1.51]	1.33 [1.30, 1.36]	1.33 [1.09, 1.64]
	Independent (1B)	1.29 [1.26, 1.31]	1.11 [1.09, 1.14]	1.58 [1.55, 1.60]	1.39 [1.36, 1.42]	–
	ReFit-interface	1.79 [1.70, 1.87]	1.27 [1.10, 1.47]	2.15 [2.13, 2.18]	1.93 [1.89, 1.98]	1.63 [1.29, 2.08]
	ReFit-full	1.85 [1.76, 1.95]	1.29 [1.12, 1.50]	2.32 [2.29, 2.35]	2.05 [2.01, 2.10]	1.77 [1.44, 2.27]
	Dedicated	2.08 [1.97, 2.19]	1.41 [1.20, 1.67]	2.94 [2.90, 2.98]	2.54 [2.43, 2.63]	2.11 [1.68, 2.75]
Nemotron-Nano	Reused	1.33 [1.27, 1.39]	1.13 [1.05, 1.20]	1.30 [1.22, 1.38]	1.36 [1.19, 1.69]	1.12 [1.00, 1.28]
	Independent (1B)	1.43 [1.38, 1.49]	1.24 [1.20, 1.27]	1.50 [1.48, 1.52]	1.50 [1.29, 1.84]	–
	ReFit-interface	1.74 [1.63, 1.86]	1.39 [1.22, 1.60]	2.07 [2.04, 2.10]	2.12 [1.82, 2.64]	1.54 [1.31, 1.86]
	ReFit-full	1.84 [1.70, 1.97]	1.42 [1.23, 1.67]	2.22 [2.18, 2.25]	2.23 [1.92, 2.75]	1.63 [1.37, 2.02]

Table 6: Wall-clock speedup over autoregressive decoding with 95% confidence intervals (vLLM, A40; three fresh server processes with three warm repetitions each).

Target	Method	SPEED-Bench bs 1	SPEED-Bench bs 8	MATH-500 bs 1	MATH-500 bs 8
R1-Distill	Reused	1.54 [1.50, 1.59]	1.23 [1.18, 1.28]	1.79 [1.76, 1.83]	1.53 [1.49, 1.57]
	ReFit-interface	1.77 [1.72, 1.82]	1.38 [1.30, 1.47]	2.10 [2.06, 2.14]	1.78 [1.73, 1.84]
	ReFit-full	1.86 [1.80, 1.92]	1.44 [1.34, 1.55]	2.26 [2.22, 2.30]	1.93 [1.87, 1.98]
Nemotron-Nano	Reused	1.55 [1.49, 1.61]	1.34 [1.29, 1.39]	1.65 [1.62, 1.68]	1.43 [1.40, 1.46]
	ReFit-interface	1.78 [1.71, 1.86]	1.56 [1.47, 1.64]	1.96 [1.92, 1.99]	1.68 [1.66, 1.71]
	ReFit-full	1.89 [1.82, 1.98]	1.62 [1.53, 1.71]	2.11 [2.07, 2.15]	1.79 [1.76, 1.83]

Table 7: DFlash wall-clock speedup over autoregressive decoding with 95% confidence intervals (128 prompts, three fresh server processes with three warm repetitions each).

Domain	R1-Distill Reuse	R1-Distill ReFit-full	R1-Distill Ded.	Nemotron-Nano Reuse	Nemotron-Nano ReFit-full
coding	1.92	2.86	3.30	1.70	2.76
humanities	1.74	2.50	2.75	1.89	2.61
math	1.94	2.86	3.36	1.71	2.68
multilingual	1.51	1.84	1.93	1.54	1.73
QA	1.66	2.53	2.81	1.68	2.44
RAG	1.87	2.68	2.94	2.14	2.76
reasoning	1.80	2.77	3.09	1.80	2.83
roleplay	1.64	2.24	2.53	1.44	1.96
STEM	1.75	2.67	2.93	1.78	2.64
summarization	1.84	2.54	2.85	1.92	2.47
writing	1.72	2.44	2.83	1.80	2.45

Table 8: Acceptance length on each SPEED-Bench domain. Every gain over reuse has a 95% confidence interval excluding zero.

History	Llama-3.1-8B n	Llama-3.1-8B EAGLE-3	Llama-3.1-8B DFlash	Qwen3-8B n	Qwen3-8B EAGLE-3	Qwen3-8B DFlash
quantization	11	1.00	0.99	23	1.00	1.00
repack	2	1.00	0.98	–	–	–
merge	3	0.98	0.96	1	1.05	1.00
CPT + merge	–	–	–	1	0.93	1.03
SFT + merge	–	–	–	1	1.08	1.01
preference + merge	–	–	–	1	0.95	0.94
abliteration	22	0.98	0.96	8	1.03	1.02
adversarial training	1	1.03	0.95	–	–	–
on-policy RL	–	–	–	4	1.01	0.98
SFT + on-policy RL	1	0.99	0.99	3	1.04	1.00
expert iteration + RL	–	–	–	1	1.24	1.03
preference opt.	9	0.95	0.95	1	0.96	0.96
SFT + preference	3	0.98	0.89	2	1.03	0.91
SFT	5	0.93	0.82	10	1.14	1.06
self-distillation	–	–	–	1	1.01	0.99
teacher distillation	2	0.83	0.83	–	–	–
SFT + teacher distill.	1	1.27	1.29	1	1.07	1.08
teacher distill. + SFT	–	–	–	2	1.09	1.04
SFT + distill. + pref.	1	0.87	0.86	–	–	–
distillation (unspecified)	–	–	–	1	2.21	1.67
undocumented	25	1.00	0.96	16	1.03	1.01

Table 9: First-token acceptance retention by family and post-training history on each derivative's own-domain prompts.